CODE QUALITY CHECK WITH SONARQUBE

WHAT IS SONARQUBE

SonarQube is a static code analysis tool that automatically analyzes source code to identify:

- Bugs (coding mistakes)
- Security vulnerabilities
- Code smells (poor coding practices)
- Duplicate code
- Test coverage
- Overall code quality

It provides a detailed report so developers can improve their code before deploying the application.

Why Do We Use SonarQube?

Suppose a developer writes 10,000 lines of Java code.

Without SonarQube:

- A senior developer has to review the code manually.
- Manual review takes a lot of time.
- Some mistakes may be missed.

With SonarQube:

- The source code is scanned automatically in a few minutes.
- It generates a report showing issues.
- Developers fix the issues before releasing the application.

SONARQUBE ARCHITECTURE.

SonarQube Server:

1. The SonarQube server is the central component of the architecture. It hosts the web application and provides the user interface for interacting with the platform.
2. The server handles HTTP requests from users and serves the SonarQube UI, dashboards, project metrics, and reports.
3. It orchestrates the analysis process, manages project configurations, and coordinates data storage and retrieval with the database.

SonarQube Database:

1. The SonarQube database is an essential component that stores various types of data required for code analysis and reporting.
2. It stores configuration settings, project metadata, analysis results, quality metrics, code issues, and other relevant information.
3. SonarQube supports multiple relational database management systems (RDBMS) such as PostgreSQL, MySQL, Microsoft SQL Server, and Oracle.

SonarQube Scanner:

1. The SonarQube scanner is a command-line tool used to analysed source code and upload analysis results to the SonarQube server.
2. It scans code repositories, identifies code issues, computes quality metrics, and generates reports based on predefined quality profiles and rules.
3. The scanner integrates with the SonarQube server to submit analysis data, which is then processed and displayed in the SonarQube UI.

KEY FEATURES

1. Static Code Analysis
2. Code Quality Metrics
3. Issue Tracking
4. Security Vulnerability Detection
5. Code Duplication Detection
6. Integration with CI/CD Pipelines

TERMS IN THE SONARQUBE

1. Code Quality

Code quality means how good your code is in terms of readability, maintainability, reliability, and security.

A good quality code should be:

- Easy to understand
- Easy to modify
- Secure
- Less error-prone
- Well-structured

2. Bug

A bug is a coding mistake that may produce incorrect results or unexpected behavior.

3. Vulnerability

A vulnerability is a security weakness that hackers can exploit.

It may allow attackers to:

- Access data
- Modify data
- Steal passwords
- Execute malicious code

4. Security Hotspot

A security hotspot is code that might be risky but requires a developer to review it.
It is not automatically a confirmed vulnerability.

5. Code Smell

A code smell is not a bug.
The application works correctly, but the code is poorly written and should be improved.

It affects:

- Maintainability
- Readability
- Future development

6. Reliability

Reliability means the software performs correctly without failures.

1. More bugs = Lower reliability.
2. Less bugs = Higher reliability
3. Example
   ATM machine should always calculate balance correctly.

7. Security

Security means protecting the application from attackers.

Examples

- SQL Injection
- Cross-Site Scripting (XSS)
- Hardcoded passwords
- Weak encryption

8. Maintainability

Maintainability means how easy it is to:

- Read code
- Modify code
- Fix bugs

- Add new features

9. Duplications

Duplicate code means the same logic is written multiple times.

12. Issues

An issue is any problem found by SonarQube.

Examples

- Bug
- Vulnerability
- Security Hotspot
- Code Smell

14. Quality Gate

A Quality Gate is a set of rules that determines whether the code passes the required quality standards.

Example rules

- Coverage ≥ 80%
- Bugs = 0
- Vulnerabilities = 0
- Code duplication < 3%

If all conditions are met:

1. Quality Gate: PASSED
   Otherwise:
2. Quality Gate: FAILED

16. Quality Profile

A Quality Profile is a collection of analysis rules used during scanning.

Example
Java Profile

- Detect SQL Injection
- Detect unused variables
- Detect duplicate code
- Detect null pointer risks

Organizations can customize which rules are enabled.

A simple way to understand SonarQube

SonarQube is like a health check-up for your source code.

- **Bugs** tell you where your code may fail.
- **Vulnerabilities** tell you where attackers could exploit your application.
- **Security Hotspots** point to code that needs a security review.
- **Code Smells** highlight code that's difficult to maintain.
- **Coverage** shows how much of your code is tested.
- **Duplications** reveal repeated code.
- **Technical Debt** estimates the effort needed to improve the code.
- **Quality Gate** decides whether the code is good enough to move to the next stage of the CI/CD pipeline.

MEMORY SETUP BEFORE INSTALLING SONARQUBE

1. sudo sysctl -w vm.max_map_count=262144
2. sudo vim /etc/sysctl.conf
   add these line ----> vm.max_map_count=262144
3. sudo sysctl -p

INSTALLATION OF SONARQUBE

1.create a SonarQube server

1. name --- SonarQube
2. ami ----- ubuntu
3. instance_type -- c71.flex.large
4. port ------ 9000
5. storage -- 12GB
6. create a instance

2.access the SonarQube

1.memory setup

1. sudo sysctl -w vm.max_map_count=262144
2. sudo vim /etc/sysctl.conf
   add these line ----> vm.max_map_count=262144
3. sudo sysctl -p

2.Install Docker

1. https://docs.docker.com/engine/install/ubuntu/
2. sudo systemctl start docker
3. sudo usermod -aG docker $USER

3.install SonarQube using docker-compose

1. sudo vim docker-compose.yml

```
version: "3"
services:
  sonarqube:
    image: sonarqube:lts-community
    depends_on:
      - db
    ports:
      - "9000:9000"
    environment:
      - SONAR_JDBC_URL=jdbc:postgresql://db:5432/sonar
      - SONAR_JDBC_USERNAME=sonar
      - SONAR_JDBC_PASSWORD=sonar
      # Limit the Java Heap to 1GB for Compute and 1GB for Web to fit in a 4GB server
      - SONAR_CE_JAVA_OPTS=-Xmx1g -Xms1g
      - SONAR_WEB_JAVA_OPTS=-Xmx1g -Xms1g
    ulimits:
      nofile:
        soft: 65536
        hard: 65536
    volumes:
      - sonarqube_data:/opt/sonarqube/data
      - sonarqube_extensions:/opt/sonarqube/extensions
      - sonarqube_logs:/opt/sonarqube/logs

  db:
    image: postgres:13
    environment:
      - POSTGRES_USER=sonar
      - POSTGRES_PASSWORD=sonar
      - POSTGRES_DB=sonar
    volumes:
      - postgresql:/var/lib/postgresql/data
    # This command is often needed for PostgreSQL containers on minimal resources
    command: postgres -c 'max_connections=200'

volumes:
  sonarqube_data:
  sonarqube_extensions:
  sonarqube_logs:
```

postgresql:

4.docker compose up -d

5.<public_ip:9000>

1. username---- admin
2. password ---- admin

1.generate token on SonarQube

1.click on profile ---- my account --- security---- tokens --- Name (sonar-token) --- type (global-analysis-token) --- expire(30) --- generate (copy to notepad)

2.Add Generated Token in Jenkins

1.manage Jenkins --- credentials --- click on global ---- choose secret-text ----- ID (sonar-token) ---- secret(paste token) ----- save

3.Configure Sonar(installations) on Jenkins

1.manage Jenkins ----- tools ----- SonarQube Scanner Installations ----- add ---- Name (sonar) ----- Install automatically ----- apply and save

4.Configure SonarQube URL in Jenkins

1.manage Jenkins ---- system ---- SonarQube servers --- click on env variables --- add SonarQube --- Name (sonar)----- Server URL (http://public_ip:9000) ---- Server authentication token (sonar-token)
